# Как устроен помощник по творчеству Расула Гамзатова

Подробное руководство для тех, кто никогда не программировал

**Как читать**

Это руководство объясняет проект с самого начала: что такое программа и нейросеть, почему нейросети ошибаются и как наш проект с этим справляется. Каждый шаг показан на **настоящих примерах** из проекта: настоящая карточка, настоящие баллы поиска, настоящие ответы нейросети. Читайте по порядку — каждая глава опирается на предыдущую. В конце есть словарик и вопросы для самопроверки.


| Глава | О чём |
|---|---|
| 1 | Самые основы: программа, Python, файлы проекта |
| 2 | Что такое нейросеть и как она пишет текст |
| 3 | Почему нейросеть «выдумывает» — настоящие примеры |
| 4 | Главная идея проекта — RAG, «экзамен с открытой книгой» |
| 5 | Шаг 1. Сбор данных (парсинг) |
| 6 | Шаг 2. Нарезка на карточки |
| 7 | Шаг 3. Поиск — три способа на одном примере |
| 8 | Шаг 4. Задание для нейросети и ответ |
| 9 | Разбор ошибок помощника |
| 10 | Шаг 5. Исследование: как проверяли и что получилось |
| 11 | Код проекта с пояснениями |
| 12 | Как запустить проект самим |
| 13 | Словарик и вопросы для самопроверки |

# 1. Самые основы

## Что такое программа

Компьютер ничего не умеет сам. Он выполняет **команды**, которые ему дали, — строго по порядку, одну за другой. Список таких команд и называется **программой**. Команды записывают на специальном языке, который понимает компьютер. Наш проект написан на языке **Python** («Пайтон»). Это один из самых простых языков, его изучают в школах.

Пример. Вот программа из одной строки:

```
print("Салам, Дагестан!")
```

Слово **print** по-английски значит «напечатать». Компьютер выполнит команду и покажет на экране: *Салам, Дагестан!* Всё остальное в проекте — это просто больше таких команд.

## Файлы и папки проекта

Программа хранится в обычном текстовом файле, название которого заканчивается на **.py** (от слова Python). Его можно открыть даже Блокнотом. Наш проект — это папка **gamzatov-rag**, в которой лежат:

| Что | Где | Зачем |
|---|---|---|
| 8 файлов-программ | config.py, step1_collect.py, step2_split.py, rag.py, step3_ask.py, app.py, step4_experiments.py, make_docs.py | каждый делает свою часть работы (подробно — в главе 11) |
| Собранные статьи | папка data/raw (20 файлов .txt) | «библиотека» — тексты о Гамзатове |
| Карточки | файл data/chunks.json | статьи, нарезанные на 303 кусочка |
| Вопросы | research/questions.json | 50 вопросов для проверки |
| Результаты | research/results, research/charts | таблицы и графики исследования |
| Документы | docs/ | PDF исследовательской работы, это руководство, видео |

## Что такое «библиотека» в программировании

Важно не перепутать. В проекте слово «библиотека» встречается в двух смыслах:

- **наша библиотека текстов** — 303 карточки о Гамзатове, в которых помощник ищет ответы;
- **библиотеки программ** — готовые чужие программы, которые мы подключаем, чтобы не писать всё самим. Например, *requests* умеет скачивать страницы из интернета, *BeautifulSoup* — разбирать их, *snowballstemmer* — находить основы русских слов, *streamlit* — делать окно в браузере. Их список лежит в файле requirements.txt.

# 2. Что такое нейросеть и как она пишет текст

Вы знаете голосового помощника **Алису**: её можно спросить о погоде или попросить рассказать сказку. Внутри Алисы, ChatGPT и нашего помощника работает **языковая модель** — программа, которая умеет писать текст. Языковую модель часто называют **нейросетью** или по-английски **LLM** (Large Language Model — «большая языковая модель»).

## Главный секрет: модель угадывает следующее слово

Вспомните клавиатуру телефона. Вы пишете «Поздравляю с днём» — и телефон предлагает: «рождения», «учителя», «победы». Он угадывает, какое слово вероятнее всего идёт дальше. Языковая модель делает **то же самое**, только во много раз лучше: она учитывает весь текст целиком, а не одно последнее слово.

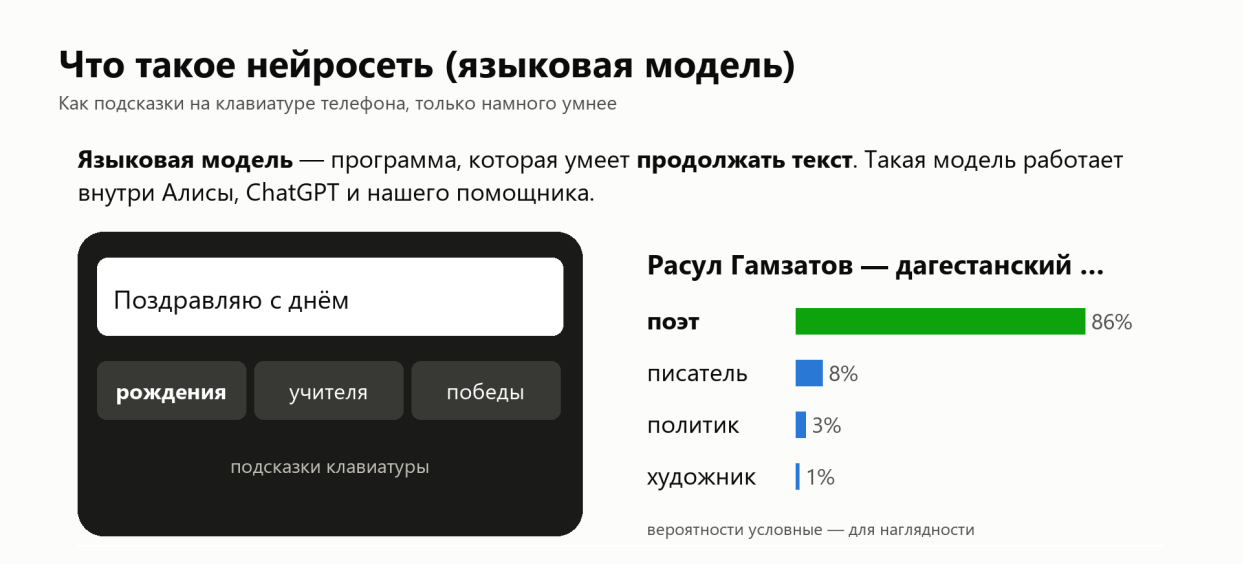



Рисунок 1. Языковая модель продолжает текст, как подсказки клавиатуры. Проценты условные — для наглядности.

Как получается целый ответ? Пример. Мы спрашиваем: «Кто такой Расул Гамзатов?» Модель угадывает первое слово ответа — «Расул». Потом, глядя на вопрос и слово «Расул», угадывает второе — «Гамзатов». Потом третье — «—», четвёртое — «дагестанский», пятое — «поэт»… Так, **слово за словом**, и пишется весь ответ. Никакого «понимания» в человеческом смысле нет — есть очень хорошее угадывание.

## Как модель научилась

Модель «обучали»: давали ей читать огромное количество текстов из интернета — миллиарды страниц — и заставляли угадывать следующее слово. Каждый раз, когда она ошибалась, её внутренние числа-настройки немного поправлялись. После миллиардов таких попыток модель стала угадывать очень хорошо.

Прочитанные тексты **не хранятся** внутри модели как книги на полке. От них остались только эти числа-настройки — их называют **параметрами**. У нашей модели **Qwen 2.5** их 1,5 миллиарда, у модели **Llama 3.2** — 3 миллиарда. Файл с моделью весит 1–2

гигабайта.

> **Самое важное**
>
> Модель **не ищет ответ в интернете** и **не открывает книгу**. Она отвечает **по памяти** — так, как «запомнила» во время обучения. Это главное, что нужно понять, чтобы разобраться в проекте.

### Где работает нейросеть в нашем проекте

Обычно нейросети работают на огромных компьютерах компаний (Алиса — у Яндекса). Мы используем бесплатную программу **Ollama** («Оллама»): она скачивает небольшую модель и запускает её **прямо на ноутбуке**, без интернета. Наши модели маленькие и слабее Алисы или ChatGPT, зато работают на обычном школьном компьютере и бесплатно.

Наша программа «разговаривает» с Ollama так: отправляет ей текст задания и получает обратно текст ответа. Как именно — покажем в главе 8.

## 3. Почему нейросеть «выдумывает»

Мы задали нашим моделям несколько вопросов о Гамзатове — просто так, без всякой библиотеки. Вот их **настоящие ответы** (сохранены в файле tutorial/assets/plain_answers.json):

| Вопрос | Модель | Ответ нейросети | Как на самом деле |
|---|---|---|---|
| Кто был режиссёром фильма «Горянка» 1975 года? | qwen2.5:1.5b | Режиссёром фильма "Горянка" 1975 года был Андрей Кончаловский. | Ирина Поплавская |
| Кто перевёл стихотворение Расула Гамзатова «Журавли» на русский язык? | qwen2.5:1.5b | Стихотворение "Журавли" Расула Гамзатова было переведено на русский язык в 1920 году. Переводчиком был известный русский поэт и переводчик Николай Александрович Бондарев. | Наум Гребнев |
| За какую книгу Расул Гамзатов получил Ленинскую премию? | qwen2.5:1.5b | Расул Гамзатов получил Ленинскую премию за книгу "Путь к свободе". | «Высокие звёзды» |
| Как звали главную героиню поэмы Расула Гамзатова «Горянка»? | qwen2.5:1.5b | Главную героиню поэмы Расула Гамзатова "Горянка" зовут Айгуль. | Асият |
| Кто был режиссёром фильма «Горянка» 1975 года? | llama3.2:3b | Режиссёром фильма "Горянка" 1975 года был Владимир Меншев. | Ирина Поплавская |

| Вопрос | Модель | Ответ нейросети | Как на самом деле |
|---|---|---|---|
| Кто перевёл стихотворение Расула Гамзатова «Журавли» на русский язык? | llama3.2:3b | Я не могу найти информацию о том, кто перевёл стихотворение Расула Гамзатова «Журавли» на русский язык. | Наум Гребнев |
| За какую книгу Расул Гамзатов получил Ленинскую премию? | llama3.2:3b | Расул Гамзатов получил Ленинскую премию за книгу "Мы" (1969). | «Высокие звёзды» |
| Как звали главную героиню поэмы Расула Гамзатова «Горянка»? | llama3.2:3b | Главную героиню поэмы Расула Гамзатова «Горянка» называют Наргиз. | Асият |

Посмотрите: почти все ответы звучат уверенно и гладко — и почти все **неправильные**. Модель придумала режиссёров (Андрей Кончаловский, Владимир Меншев), героинь (Айгуль, Наргиз), книги («Путь к свободе», «Мы») и даже переводчика «Николая Бондарева» с годом перевода 1920 — за три года до рождения Гамзатова! Только один ответ из восьми честный: «Я не могу найти информацию».

**Галлюцинация**

Такой уверенный, правдоподобный, но неверный ответ называется **галлюцинацией**. Причина простая: модель обучена писать **гладкий** текст, а не **проверенный**. Если она мало читала о теме — а о дагестанских поэтах в интернете написано гораздо меньше, чем, например, о Пушкине, — она всё равно напишет что-то «похожее на правду».

Для школьника это опасно: можно выучить неправильный ответ, даже не заметив подвоха. Наш проект решает эту проблему.

## 4. Главная идея проекта — RAG

Представьте два экзамена по литературе.

- **Экзамен по памяти.** Ученик отвечает только то, что помнит. Если не помнит — может честно сказать «не знаю», а может и присочинить. Так работает обычная нейросеть.
- **Экзамен с открытой книгой.** Разрешили пользоваться учебником. Теперь не нужно помнить всё — нужно **найти правильную страницу** и прочитать ответ. Так работает наш помощник.




Рисунок 2. Идея проекта: дать нейросети «учебник»

По-научному такой подход называется **RAG** (читается «раг») — от английского *Retrieval-Augmented Generation*, «генерация, дополненная поиском». Его придумали исследователи в 2020 году. Суть в двух словах:

**RAG в двух шагах**

1) по вопросу **найти** в библиотеке несколько подходящих кусочков текста;
2) отдать их нейросети вместе с вопросом и попросить **ответить только по ним**.

Значит, в проекте две главные «роли»:

- **библиотекарь** — часть программы, которая ищет нужные кусочки текста (это обычная программа, не нейросеть);
- **рассказчик** — нейросеть, которая читает найденное и пишет ответ.

А чтобы библиотекарю было где искать, сначала нужно собрать библиотеку и разложить её по «карточкам». Получается пять шагов:




Рисунок 3. Пять шагов проекта. Каждый шаг делает свой файл программы.

# 5. Шаг 1. Сбор данных (парсинг)

**Файл:** step1_collect.py. **Результат:** папка data/raw с 20 текстовыми файлами.

## Откуда берём тексты

Мы выбрали **Википедию** и **Викицитатник** (сборник цитат). Причины:

- там есть статьи и о самом Гамзатове, и о его произведениях («Журавли», «Горянка», «Мой Дагестан», «Сказание о Хочбаре», «Пожелание»), о его отце Гамзате Цадасе, о родном селе Цада, о переводчиках Науме Гребневе и Якове Козловском, о памятниках «Белые журавли»;
- тексты Википедии распространяются по свободной лицензии **CC BY-SA** — их можно законно использовать;
- стихи самого Гамзатова защищены авторским правом (оно действует 70 лет после смерти автора), поэтому сборники его стихов мы целиком не копировали. В статьях Википедии есть только короткие цитаты.

| № | Статья | Символов |
|---|---|---|
| 1 | Гамзатов, Расул Гамзатович | 24173 |
| 2 | Расул Гамзатович Гамзатов | 1613 |
| 3 | Журавли (песня) | 19822 |
| 4 | Мой Дагестан (книга) | 6137 |
| 5 | Горянка (поэма) | 6327 |
| 6 | Горянка (фильм) | 5205 |
| 7 | Сказание о Хочбаре | 5157 |
| 8 | Хочбар (опера) | 2089 |
| 9 | Пожелание (песня) | 4414 |
| 10 | Цадаса, Гамзат | 5844 |
| 11 | Цада | 2364 |
| 12 | Белые журавли (памятник) | 1011 |
| 13 | Белые журавли (музей) | 2015 |
| 14 | Гамзатов, Гаджи Гамзатович | 8621 |
| 15 | Гамзатова, Патимат Расуловна | 2331 |
| 16 | Дагестанский музей изобразительных искусств имени П. С. Гамзатовой | 3761 |
| 17 | Гребнев, Наум Исаевич | 31471 |
| 18 | Козловский, Яков Абрамович | 9452 |
| 19 | Аварская литература | 4101 |
| 20 | Дагестанская литература | 4645 |

## Что такое HTML и зачем парсинг

Страница сайта, которую вы видите в браузере, на самом деле записана на языке разметки **HTML**. В нём текст перемешан со служебными пометками в угловых скобках — **тегами**. Вот настоящее начало статьи о фильме «Горянка» (адреса ссылок сокращены):

```
<p><b>«Горя´нка»</b> — советский художественный фильм <a href="/wiki/Поплавская,_Ирина"
   title="Поплавская, Ирина Ивановна">Ирины Поплавской</a>, основанный на сюжете поэмы
<a href="/wiki/Гамзатов,_Расул" title="Гамзатов, Расул Гамзатович">Расула Гамзатова</a>
«<a href="/wiki/Горянка_(поэма)">Горянка</a>» о дагестанской девушке, …</p>
```

Тег ** ** означает «абзац» (paragraph), **<a>** — ссылку, **<b>** — жирный шрифт. Каждый тег открывается ( ) и закрывается ( ). **Парсинг** — это разбор такой страницы: программа находит нужные теги и вытаскивает из них чистый текст:

**Что получилось после парсинга**

«Горянка» — советский художественный фильм Ирины Поплавской, основанный на сюжете поэмы Расула Гамзатова «Горянка» о дагестанской девушке, бросившей вызов традиционным устоям и отказавшейся выйти замуж за нелюбимого человека.

## Что делает программа — по шагам

- Берёт из файла config.py список из 20 статей.
- Для каждой статьи просит у Википедии HTML-код (с помощью библиотеки *requests*).
- Выбрасывает мусор: номера примечаний [1], кнопки «править», навигационные таблицы.
- Проходит по странице сверху вниз (библиотека *BeautifulSoup*) и превращает в строки текста заголовки, абзацы, списки и таблицы. Строка таблицы «Режиссёр | Ирина Поплавская» превращается в «Режиссёр: Ирина Поплавская».
- Выбрасывает служебные разделы: «Примечания», «Литература», «Ссылки».
- Сохраняет текст в файл, например *data/raw/06_Горянка_фильм.txt*, и ждёт 3 секунды перед следующей статьёй, чтобы не перегружать сайт.

**Интересная деталь для выступления**

Сначала мы использовали упрощённый способ получения текста, но оказалось, что он **теряет списки** — например, полный перечень произведений Гамзатова и все цитаты. Пришлось переписать программу на разбор HTML. А ещё Википедия однажды ответила «Слишком много запросов, подождите 21 секунду» — программа научилась ждать и повторять. В исследованиях так бывает всегда: данные приходится проверять.

**Итог шага 1:** 20 статей, около 150 000 символов — это примерно 80 страниц книги.

# 6. Шаг 2. Нарезка на карточки

**Файл:** step2_split.py. **Результат:** файл data/chunks.json с 303 карточками.

## Зачем резать

- Нейросеть может прочитать за один раз только ограниченный объём текста. Мы разрешаем ей 4096 «токенов» (о токенах — в главе 8) — это несколько страниц. Вся библиотека в 80 страниц туда не влезет.
- Маленькой нейросети трудно найти один факт в длинной статье. Чем короче кусочек, тем легче.
- Искать удобнее по маленьким кусочкам: так библиотекарь находит именно тот абзац, где ответ.

## Как режем

Аккуратно, **по абзацам**, чтобы не разрезать мысль посередине. Программа берёт абзацы по очереди и складывает их в карточку, пока её длина не превысит 700 символов (примерно полстраницы). Тогда начинается новая карточка. Если один абзац сам длиннее 700 символов, его режут по предложениям. Кроме того, программа помнит, в каком разделе статьи находится абзац.

Вот **настоящая** карточка № 116 так, как она записана в файле data/chunks.json:


```
{
 "id": 116,
 "source": "Горянка (фильм)",
 "section": "Начало статьи",
 "url": "https://ru.wikipedia.org/wiki/Горянка_(фильм)",
 "text": "Горянка\nЖанр: драма\nРежиссёр: Ирина Поплавская\nАвторы сценария: Расул Гамзатов
   Ирина Поплавская\nВ главных ролях: Татьяна Шумова Ислам Казиев Рамаз
   Гиоргобиани\nОператор: Анатолий Николаев\nКомпозитор: Мурад Кажлаев\nКинокомпании:
   Мосфильм, Первое творческое объединение\nДлительность: 63 мин.\nСтрана: СССР\nЯзык:
   русский\nГод: 1975\nIMDb: ID 5364916\n«Горя´нка» — советский художественный…"
}
```


Здесь **id** — номер карточки, **source** — из какой статьи она взята, **section** — раздел статьи, **url** — адрес страницы, **text** — сам текст (знак \n означает переход на новую строку). Благодаря source и url помощник потом может показать, **откуда** взят ответ.

**Итог шага 2:** 303 карточки, средняя длина — около 500 символов.

# 7. Шаг 3. Поиск: как работает «библиотекарь»

**Файл:** rag.py, часть *Library*. Это самая важная глава — читайте не спеша.

Пользователь спрашивает: **«Кто был режиссёром фильма «Горянка» 1975 года?»** У нас 303 карточки. Нужно выбрать 3, в которых, скорее всего, есть ответ. Компьютер **не понимает смысла** слов. Зато он отлично умеет **сравнивать**: есть ли одинаковые слова в вопросе и в карточке. На этом и построен поиск. Мы попробовали три способа, каждый следующий умнее предыдущего.

## Подготовка вопроса

Сначала программа приводит вопрос к удобному виду:

- делает все буквы маленькими: «Горянка» → «горянка»;
- заменяет «ё» на «е»: «режиссёром» → «режиссером» (в одних текстах пишут ё, в других — е);
- разбивает текст на слова, выкидывая знаки препинания и кавычки;
- выбрасывает **стоп-слова** — очень частые слова, которые ничего не говорят о смысле: «кто», «был», «какой», «в», «на», «и», «что», «где», «когда» и т. п. (полный список — в начале файла rag.py).

Из вопроса «Кто был режиссёром фильма «Горянка» 1975 года?» остаётся пять слов: **режиссером, фильма, горянка, 1975, года**.

## Способ 1. Считаем совпадающие слова

Для каждой карточки считаем, сколько из этих пяти слов в ней встречается. Одно совпадение — один балл. Настоящий результат для нашего вопроса (три лучшие карточки):

| Карточка | Какие слова совпали | Баллы |
|---|---|---|
| № 116 «Горянка (фильм)» — Начало статьи | фильма, горянка, 1975, года | 4 |
| № 111 «Горянка (поэма)» — Экранизации | фильма, горянка, 1975 | 3 |
| № 281 «Козловский, Яков Абрамович» — Переводы Я. Козловского | горянка, 1975, года | 3 |

Неплохо, но есть проблема. Компьютер сравнивает слова **буква в букву**. Для него «горянка» и «горянке», «фильма», «фильме» и «фильм» — совсем разные слова. Слово «режиссером» в вопросе не совпало со словом «режиссер» в карточке № 116! А в русском языке у каждого слова много форм (падежи, числа).

## Способ 2. Сравниваем основы слов

Решение: перед сравнением отрезать у каждого слова **окончание** и оставить **основу**. Это называется **стемминг**. Мы используем готовую программу-стеммер **Snowball** («Сноубол»): она знает правила русских окончаний. Настоящие примеры её работы:

| Слово | Основа после стемминга |
|---|---|
| режиссёром | режиссер |
| фильма | фильм |
| фильме | фильм |
| горянка | горянк |
| горянке | горянк |
| журавли | журавл |
| журавлей | журавл |
| журавлях | журавл |
| Гамзатова | гамзатов |
| Гамзатову | гамзатов |

Теперь карточка № 116 совпадает по всем пяти основам и получает **5 баллов из 5**. Поиск стал точнее.

## Способ 3. BM25: редкие слова важнее

Остаётся ещё одна проблема: пока все слова стоят одинаково — по одному баллу. Но посмотрите, в скольких карточках встречается каждая основа нашего вопроса:

| Основа | В скольких карточках из 303 | Вес слова в BM25 |
|---|---|---|
| режиссер | 9 | 3.47 |
| 1975 | 12 | 3.19 |
| фильм | 19 | 2.75 |
| горянк | 30 | 2.30 |
| год | 79 | 1.34 |

Основа «год» встречается в 79 карточках — почти в каждой четвёртой. Она плохо помогает найти нужную. А «режиссер» — всего в 9 карточках: если в вопросе есть это слово, нам нужна именно одна из этих девяти. **Редкое слово — сильная подсказка.**

Способ **BM25** (так называется формула, «Best Matching 25» — «лучшее совпадение, вариант 25») даёт каждому слову **вес**: чем реже слово, тем больше вес. Вес «режиссер» — 3,47, а вес «год» — всего 1,34. Баллы карточки — это сумма весов совпавших слов, с двумя поправками:

- **Повторы.** Если слово встречается в карточке несколько раз, баллов становится больше, но каждый следующий повтор добавляет всё меньше. Иначе карточка, где

слово «фильм» написано десять раз, победила бы только за это.

- **Длина.** Длинная карточка содержит больше слов и чаще что-то совпадает случайно. Поэтому у длинных карточек баллы немного уменьшаются, у коротких — немного увеличиваются.

**Для любопытных: формула BM25 из файла rag.py**

баллы = сумма по словам вопроса: вес_слова × (повторы × 2,5) / (повторы + 1,5 × (0,25 + 0,75 × длина / средняя_длина))
вес_слова = ln(1 + (303 − карточек_со_словом + 0,5) / (карточек_со_словом + 0,5))
Например, для «режиссер»: ln(1 + (303 − 9 + 0,5)/(9 + 0,5)) = ln(32) ≈ 3,47.

Настоящий результат BM25 для нашего вопроса — эти три карточки и получит нейросеть:

| Место | Карточка | Баллы | Есть ли там ответ? |
|---|---|---|---|
| 1 | № 111 «Горянка (поэма)» — Экранизации | 15.07 | да: «режиссёр Ирина Поплавская» |
| 2 | № 116 «Горянка (фильм)» — Начало статьи | 14.98 | да: «режиссёр Ирина Поплавская» |
| 3 | № 125 «Горянка (фильм)» — Проблематика | 11.20 | нет (но про того же режиссёра) |

## Какой способ лучше — проверили на 45 вопросах

Для каждого из 45 обычных вопросов мы проверили: есть ли правильный ответ хотя бы в одной из найденных карточек (1, 3 или 5 штук).

| Способ поиска | 1 карточка | 3 карточки | 5 карточек |
|---|---|---|---|
| 1. Совпадение слов | 76% | 84% | 89% |
| 2. Основы слов | 73% | 89% | 96% |
| 3. BM25 | 76% | 89% | 93% |

Вывод: основы слов сильно помогают — при 5 карточках находится 96% ответов против 89%. Когда берём только одну карточку, все способы почти равны: нужная карточка часто оказывается второй или третьей. Поэтому мы даём нейросети **три** карточки — это золотая середина: ответ обычно находится, а читать нейросети не слишком много (на слабом ноутбуке каждая карточка добавляет секунды ожидания).

## А что такое «поиск по смыслу» (векторный поиск)?

Этого в нашем проекте **нет**, но о нём часто спрашивают. Наш поиск сравнивает слова. Если спросить «Кто **снял** фильм „Горянка“?», слово «снял» не совпадёт со словом «режиссёр», хотя смысл один.

Поиск по смыслу использует ещё одну нейросеть, которая превращает каждый текст в набор чисел — **вектор**, что-то вроде «координат смысла» на карте. Тексты с похожим

смыслом получают близкие координаты: «снял фильм» и «режиссёр фильма» окажутся рядом, даже если слова разные. Мы выбрали поиск по словам, потому что его легко понять и проверить вручную, он быстрый и не требует второй нейросети. Поиск по смыслу — хорошая идея для развития проекта.

# 8. Шаг 4. Задание для нейросети и ответ

**Файл:** rag.py, функции *answer* и *ask_model*.

## Промпт — текст задания

Найденные карточки и вопрос программа вставляет в заранее написанный шаблон. Получившийся текст задания называется **промптом** (от англ. prompt — «подсказка»). Он состоит из двух частей.

**1. Роль** (так называемое «системное сообщение»): кем быть нейросети.

```
Ты — помощник школьника по творчеству Расула Гамзатова. Отвечай по-русски, коротко: 1–3
  предложения.
```

**2. Само задание**. Вот настоящий промпт для нашего вопроса (тексты карточек сокращены):

```
Ниже даны фрагменты из библиотеки. Ответь на вопрос, используя ТОЛЬКО эти фрагменты.
Если в них нет ответа, скажи: «В моей библиотеке нет ответа».

[1] Горянка (поэма) — Экранизации:
В 1975 году по поэме был снят фильм «Горянка» (режиссёр Ирина Поплавская). В фильме звучат
  отрывки из поэмы в русском переводе в исполнении Сергея Бон…

[2] Горянка (фильм) — Начало статьи:
Горянка
Жанр: драма
Режиссёр: Ирина Поплавская
Авторы сценария: Расул Гамзатов Ирина Поплавская
В главных ролях: Татьяна Шумова Ислам Казиев Рамаз Гио…

[3] Горянка (фильм) — Проблематика:
По словам режиссёра, в фильме её «волновала не столько проблема закрепощения и раскрепощения
   дагестанской женщины», но скорее более общая тема — хотел…

Вопрос: Кто был режиссёром фильма «Горянка» 1975 года?
```

Обратите внимание на два важных места:

- слово **ТОЛЬКО** — нейросеть должна отвечать только по фрагментам, а не «по памяти»;
- разрешение сказать **«В моей библиотеке нет ответа»** — если не дать нейросети такой возможности, она будет стараться ответить хоть что-нибудь и начнёт выдумывать.

## Настройки нейросети

| Настройка | Значение | Что значит |
|---|---|---|
| model | qwen2.5:1.5b | какую нейросеть использовать |
| temperature | 0 | «температура» — сколько случайности при выборе следующего слова. При 0 модель всегда берёт самое вероятное слово, поэтому на один и тот же вопрос отвечает одинаково. Для честного эксперимента это важно |
| num_predict | 200 | ответ не длиннее 200 токенов |
| num_ctx | 4096 | сколько токенов нейросеть может прочитать за раз (вопрос + карточки + ответ) |

**Токен** — кусочек текста, которым «думает» нейросеть. Это не всегда целое слово: длинные и редкие слова делятся на несколько частей, короткие частые слова — один токен. В среднем один токен русского текста — это 3–4 буквы. Наш промпт с тремя карточками занимает около 700–800 токенов.

## Ответ

Программа отправляет промпт в Ollama и ждёт. На нашем ноутбуке ответ с библиотекой занимает в среднем 21.0 секунды, без библиотеки — 8.8 (нейросети не нужно читать карточки). Нейросеть ответила: **«Ирина Поплавская»** — верно! Помощник показывает ответ и под ним ссылки на три карточки-источника.

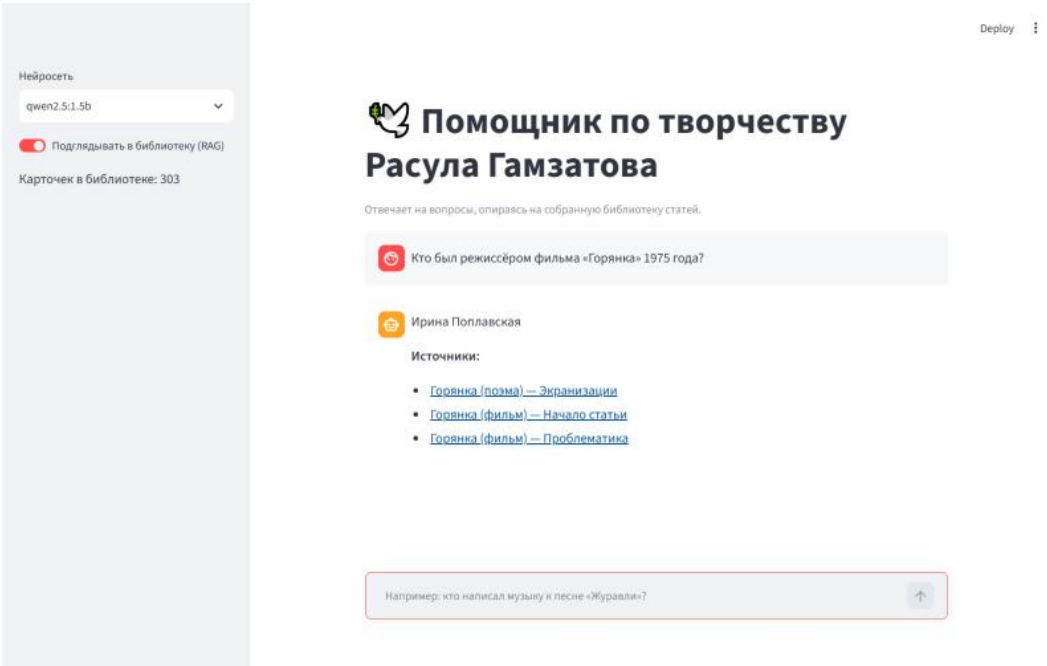



Рисунок 4. Настоящий снимок окна помощника (файл app.py): ответ и источники

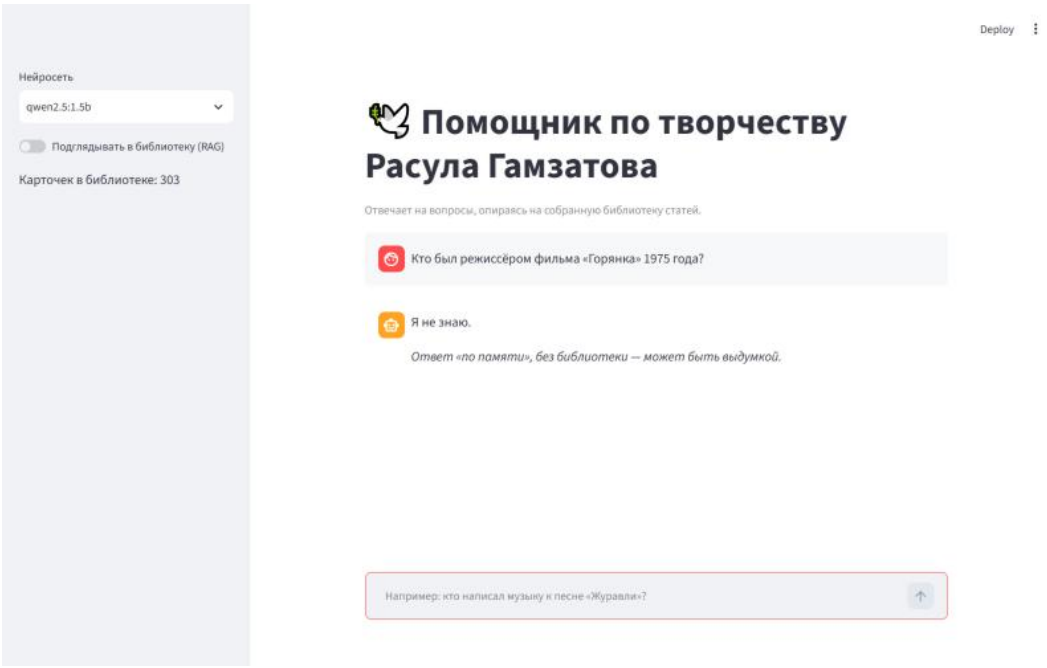



Рисунок 5. Тот же вопрос с выключенным переключателем «Подглядывать в библиотеку»

Без библиотеки та же нейросеть ответила «Я не знаю». Почему не выдумала, как в главе 3? Потому что в нашем шаблоне без библиотеки тоже есть фраза «Если ты не знаешь точного ответа, так и скажи: „Я не знаю“». А в главе 3 мы задавали вопрос **без этой фразы** — и нейросеть сочиняла. Это ещё раз показывает, как сильно ответ зависит от того, как составлено задание.

# 9. Разбор ошибок помощника

Помощник не идеален. Понимать, **почему** он ошибается, не менее важно, чем знать, как он работает. Все примеры — настоящие ответы из эксперимента.

## Ошибка 1. «Журавли» перевёл… Козловский?

Вопрос: **Кто перевёл стихотворение «Журавли» на русский язык?** Ответ помощника: **Яков Козловский перевёл стихотворение «Журавли» на русский язык.** Правильно: **Наум Гребнев**.

Смотрим, что нашёл библиотекарь: Журавли (песня) — История создания; Горянка (поэма) — История; Журавли (песня) — История создания.

Первая карточка правильная: в ней Гамзатов пишет «Мой друг Наум Гребнев превосходно перевёл „Журавлей“ на русский язык». Но вторая карточка — из статьи о поэме «Горянка»: «На русский язык поэму **перевёл** … **Яков Козловский**». Слово «перевёл» очень редкое (всего в 4 карточках), поэтому BM25 дал этой карточке много баллов. Маленькая нейросеть увидела в двух карточках два разных переводчика и выбрала не того.

### Ошибка 2. Певец или композитор?

Вопрос: **Какой певец первым исполнил песню «Журавли»?** Ответ: **Ян Френкель.** Правильно: **Марк Бернес**. Карточки были правильные (все три — из статьи «Журавли (песня)»), но нейросеть перепутала композитора (Ян Френкель написал музыку) с певцом. Это ошибка «рассказчика», а не библиотекаря.

### Ошибка 3. Нужная карточка не нашлась

Вопрос: **Как зовут главную героиню поэмы «Горянка»?** Ответ: **Горянка.** Правильно: **Асият**. Все три найденные карточки — из раздела «Театральные постановки», где имя героини не упоминается. Слова вопроса «главную героиню поэмы Горянка» совпали с текстом о спектаклях. Это ошибка библиотекаря.

**Вывод**

Ошибки бывают двух видов: **библиотекарь** нашёл не те карточки, или **рассказчик** (маленькая нейросеть) неправильно прочитал правильные карточки. Поэтому помощник всегда показывает источники — ответ можно проверить за минуту.

## 10. Шаг 5. Исследование: как проверяли и что получилось

**Файл:** step4_experiments.py. **Вопросы:** research/questions.json. **Результаты:** research/results.

### Гипотеза

**Гипотеза**

Небольшая нейросеть без доступа к источникам плохо отвечает на вопросы о Гамзатове (ошибается или не знает); если дать ей библиотеку (RAG), доля верных ответов заметно вырастет.

### Вопросы

Мы составили 50 вопросов с правильными ответами, причём только такие, ответ на которые **точно есть** в собранных статьях (иначе нечестно спрашивать):

- **45 обычных** вопросов по пяти темам: биография, награды, «Журавли», произведения, память о поэте;
- **5 вопросов-ловушек** с ошибкой внутри. Например: «За какой роман Расул Гамзатов получил Нобелевскую премию?» — он её не получал. Правильно ответить «нет» или «не знаю». Ловушки проверяют, не выдумывает ли нейросеть.

Так выглядит один вопрос в файле:

```
{"id": 1, "topic": "Биография", "question": "В каком году родился Расул Гамзатов?",
  "answer": "В 1923 году (8 сентября)", "keywords": ["1923"]}
```

## Как проверяли ответы

Проверяла программа — по **ключевым словам** (поле keywords). Ответ засчитывается как **верный**, если в нём есть ключевое слово. Если нейросеть написала «не знаю» или «нет ответа» — это **«не знаю»**. Если ни то, ни другое — это **ошибка**. Для ловушек верно — отказаться или сказать «нет».

| Вопрос | Ответ нейросети | Ключевое слово | Итог |
|---|---|---|---|
| В каком году родился Расул Гамзатов? | 8 сентября 1923 года. | 1923 | верно |
| Кто был режиссёром фильма «Горянка»? | Ирина Поплавская | поплавск | верно (основа подходит к любой форме) |
| Кто перевёл «Журавли»? | Яков Козловский перевёл… | гребнев | ошибка |
| За какой роман Нобелевская премия? | В моей библиотеке нет ответа. | — | верно (ловушка распознана) |

**Честно о недостатках**

Автоматическая проверка иногда ошибается. Например, ответ «За книгу „Высокие звёзд“» (с опечаткой) сначала не засчитался, а ответ на ловушку «Вопрос не имеет ответа» был ошибочно записан в ошибки. Мы нашли это при ручной проверке и исправили правила. Поэтому все ответы стоит перепроверить глазами — они есть в Приложении 3 исследовательской работы.

## Результаты

Полный эксперимент (50 вопросов × 2 модели × 2 режима = 200 ответов) на нашем ноутбуке длился бы несколько часов. Поэтому нейросети задали **15 вопросов**: 12 обычных и 3 ловушки, модель qwen2.5:1.5b. Поиск проверили на всех 45 обычных вопросах (глава 7).

| Режим | Верно | «Не знаю» | Ошибка | Ловушки распознаны | Время ответа |
|---|---|---|---|---|---|
| Без библиотеки | 0 из 12 | 12 | 0 | 3 из 3 | 8.8 с |
| С библиотекой (RAG) | 7 из 12 | 0 | 5 | 3 из 3 | 21.0 с |

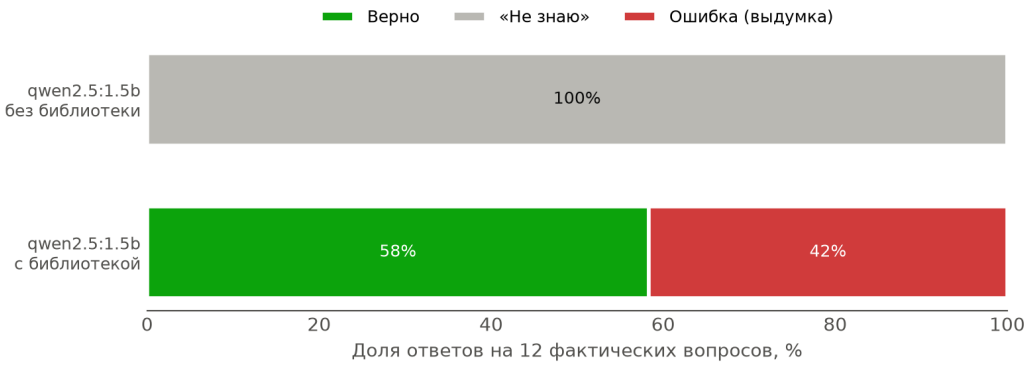



Рисунок 6. Ответы нейросети без библиотеки и с библиотекой

Как это понимать:

- **Без библиотеки** модель на все 12 вопросов ответила «Я не знаю» — потому что мы разрешили ей так говорить. Это честно, но бесполезно. А если не разрешать (глава 3) — она выдумывает.
- **С библиотекой** верных ответов стало 7 из 12, и все три ловушки распознаны. **Гипотеза подтвердилась.**
- Но 5 ответов — ошибки (глава 9). Маленькой модели не хватает «внимательности». Улучшить можно более мощной моделью и более умным поиском.
- 12 вопросов — это немного, поэтому цифры примерные. Программа умеет повторить полный опыт одной командой.

# 11. Код проекта с пояснениями

Здесь — самые важные куски кода с объяснением построчно. Не обязательно понимать каждую запятую: главное — понимать, **что** делает каждая часть. Текст после знака # — это комментарий: подсказка для человека, компьютер её пропускает.

## config.py — настройки

Все настройки в одном месте: какие статьи скачивать, длина карточки, сколько карточек давать нейросети, какую модель использовать. Чтобы добавить новую статью в библиотеку, достаточно дописать строку в список SOURCES и заново запустить шаги 1 и 2.

```
    ("ru.wikipedia.org", "Цада"),
    ("ru.wikipedia.org", "Белые журавли (памятник)"),
    ("ru.wikipedia.org", "Белые журавли (музей)"),
    ("ru.wikipedia.org", "Гамзатов, Гаджи Гамзатович"),
    ("ru.wikipedia.org", "Гамзатова, Патимат Расуловна"),
    ("ru.wikipedia.org", "Дагестанский музей изобразительных искусств имени П. С.
      Гамзатовой"),
    ("ru.wikipedia.org", "Гребнев, Наум Исаевич"),
    ("ru.wikipedia.org", "Козловский, Яков Абрамович"),
    ("ru.wikipedia.org", "Аварская литература"),
    ("ru.wikipedia.org", "Дагестанская литература"),
]

# Нарезка текста на карточки
CHUNK_SIZE = 700  # примерная длина одной карточки в символах

# Поиск
TOP_K = 3  # сколько карточек отдавать нейросети

# Нейросеть (запускается локально через программу Ollama)
OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL = "qwen2.5:1.5b"
MODELS_FOR_EXPERIMENT = ["qwen2.5:1.5b", "llama3.2:3b"]
```

## rag.py — подготовка слов

```
def words_of(text):
    """Разбивает текст на слова: маленькие буквы, «ё» → «е», без стоп-слов."""
    words = re.findall(r"[а-яёa-z0-9]+", text.lower().replace("ё", "е"))
    return [w for w in words if w not in STOP_WORDS]

def stems_of(text):
    """То же, но каждое слово обрезано до основы: «журавлях» → «журавл»."""
    return [_stemmer.stemWord(w) for w in words_of(text)]
```

- **def words_of(text):** — объявляем «функцию» (мини-программу) с именем words_of, которая получает текст;
- **text.lower().replace("ё", "е")** — делаем буквы маленькими и меняем ё на е;
- **re.findall(r"[а-яёa-z0-9]+", …)** — находим все слова: подряд идущие буквы и цифры;
- **[w for w in words if w not in STOP_WORDS]** — оставляем только слова, которых нет в списке стоп-слов;
- **stems_of** делает то же самое и вдобавок обрезает каждое слово до основы стеммером.

## rag.py — поиск (способ BM25)

```
        if method == "bm25":
            # Способ 3: BM25 — редкие слова важнее частых,
            # а длинная карточка не получает преимущества только за длину
            k1, b = 1.5, 0.75
            q = set(stems_of(question))
            scores = []
            for counts, stems in zip(self.counts, self.stem_lists):
                norm = k1 * (1 - b + b * len(stems) / self.avg_len)
                scores.append(sum(
                    self.idf.get(s, 0) * counts[s] * (k1 + 1) / (counts[s] + norm)
                    for s in q if s in counts))
            return scores
```

- **k1, b = 1.5, 0.75** — две настройки формулы: насколько быстро «устают» повторы и насколько сильно учитывается длина карточки. Это стандартные значения;
- **for counts, stems in zip(…)** — перебираем все 303 карточки по очереди;
- **norm = …** — поправка на длину карточки;
- **sum(self.idf[s] * …)** — складываем вес каждого слова вопроса, которое есть в карточке.

```
    def search(self, question, k=TOP_K, method="bm25"):
        """Возвращает k самых подходящих карточек."""
        scores = self._score(question, method)
        best = sorted(range(len(scores)), key=lambda i: -scores[i])[:k]
        return [self.chunks[i] for i in best if scores[i] > 0]
```

Функция **search** считает баллы для всех карточек, сортирует их от большего к меньшему и возвращает первые k (у нас 3).

## rag.py — запрос к нейросети

```
def ask_model(prompt, model=MODEL):
    """Отправляет запрос нейросети, запущенной в Ollama, и возвращает её ответ."""
    response = requests.post(OLLAMA_URL, json={
        "model": model,
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": prompt},
        ],
        "stream": False,
        # temperature = 0: на один и тот же вопрос модель отвечает одинаково
        "options": {"temperature": 0, "num_predict": 200, "num_ctx": 4096},
    }, timeout=900)
    response.raise_for_status()
    return response.json()["message"]["content"].strip()
```

- **requests.post(OLLAMA_URL, json={…})** — отправляем задание в программу Ollama (она «слушает» на адресе http://localhost:11434 — localhost значит «этот же компьютер»);
- **messages** — переписка: сначала роль (system), потом задание (user);
- **options** — настройки из главы 8;
- **response.json()["message"]["content"]** — достаём из ответа Ollama сам текст.

```
def answer(question, library=None, model=MODEL, k=TOP_K):
    """Отвечает на вопрос. Если library=None — нейросеть отвечает «по памяти»."""
    if library is None:
        return ask_model(PROMPT_NO_LIBRARY.format(question=question), model), []
    found = library.search(question, k=k)
    fragments = "\n\n".join(
        f"[{i}] {c['source']} — {c['section']}:\n{c['text']}" for i, c in enumerate(found,
          1))
    prompt = PROMPT_WITH_LIBRARY.format(fragments=fragments, question=question)
    return ask_model(prompt, model), found
```

Функция **answer** соединяет всё: если библиотеки нет — отправляет вопрос «по памяти»; если есть — ищет карточки, вставляет их в шаблон и отправляет. Возвращает ответ и найденные карточки (для ссылок).

## step2_split.py — нарезка

```
def make_chunks(text, size):
    """Складывает абзацы в карточки, пока карточка не станет длиннее size."""
    pieces = []
    for paragraph in text.split("\n"):
        paragraph = paragraph.strip()
        if not paragraph:
            continue
        # Слишком длинный абзац режем по предложениям
        pieces.extend(split_into_sentences(paragraph) if len(paragraph) > size else
          [paragraph])

    chunks, current = [], ""
    for piece in pieces:
        if current and len(current) + len(piece) > size:
            chunks.append(current)
            current = ""
        current = f"{current}\n{piece}" if current else piece
    if current:
        chunks.append(current)
    return chunks
```

Сначала абзацы (а слишком длинные — предложения) складываются в список pieces. Потом они по очереди добавляются в текущую карточку current; если следующий кусок не влезает в size символов — карточка закрывается и начинается новая.

## step1_collect.py — главный цикл сбора

```
def main():
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    catalog = []
    for number, (site, title) in enumerate(SOURCES, start=1):
        html = download_html(site, title)
        if not html:
            print(f"[!] Не найдено: {title}")
            continue
        text = parse_article(html)
        url = f"https://{site}/wiki/{title.replace(' ', '_')}"
        safe_title = re.sub(r"[^\w]+", "_", title).strip("_")
        file_name = f"{number:02d}_{safe_title}.txt"
        (RAW_DIR / file_name).write_text(f"{title}\n{url}\n\n{text}", encoding="utf-8")
        catalog.append({"file": file_name, "title": title, "url": url, "chars": len(text)})
        print(f"[ok] {title}: {len(text)} символов")
        time.sleep(3)  # вежливая пауза, чтобы не нагружать сайт

    (RAW_DIR.parent / "sources.json").write_text(
        json.dumps(catalog, ensure_ascii=False, indent=2), encoding="utf-8")
    total = sum(item["chars"] for item in catalog)
    print(f"\nГотово: {len(catalog)} статей, всего {total} символов.")
```

# 12. Как запустить проект самим

Всё делается на Windows. Понадобится интернет только для установки.

## Установка (один раз)

- Скачать и установить **Python 3.11** с сайта python.org. При установке поставить галочку *Add Python to PATH*.
- Скачать и установить **Ollama** с сайта ollama.com. После установки она работает в фоне (значок у часов).
- Открыть папку проекта, в адресной строке Проводника написать **cmd** и нажать Enter — откроется командная строка (чёрное окно для команд).
- Скачать нейросеть (около 1 ГБ):

```
ollama pull qwen2.5:1.5b
```

- Установить библиотеки программ:

```
pip install -r requirements.txt
```

## Запуск помощника

```
streamlit run app.py
```

Откроется страница в браузере. Слева — выбор нейросети и переключатель «Подглядывать в библиотеку». Внизу — поле для вопроса. Первый ответ может идти около минуты (нейросеть загружается), потом быстрее.

### Другие команды

| Команда | Что делает | Сколько длится |
|---|---|---|
| python step1_collect.py | заново скачать статьи | ≈ 2 минуты |
| python step2_split.py | заново нарезать карточки | секунды |
| python step3_ask.py | помощник в командной строке | — |
| python step4_experiments.py search | эксперимент с поиском | секунды |
| python step4_experiments.py --quick | быстрый эксперимент (15 вопросов) | ≈ 10–15 минут |
| python step4_experiments.py | полный эксперимент (200 ответов) | несколько часов |
| python make_docs.py | пересобрать PDF исследовательской работы | секунды |

**Если что-то не работает**

Если пишет «ConnectionError» — не запущена Ollama (запустите её из меню Пуск). Если всё очень медленно — закройте лишние программы: нейросети нужна оперативная память.

## 13. Словарик

| Слово | Простыми словами |
|---|---|
| Программа | список команд для компьютера |
| Python | язык программирования, на котором написан проект |
| Нейросеть, языковая модель, LLM | программа, которая пишет текст, угадывая следующее слово |
| Параметры | миллиарды чисел-настроек внутри нейросети — её «память» |
| Обучение | процесс, когда нейросеть читает тексты и подстраивает параметры |
| Галлюцинация | уверенный, но выдуманный ответ нейросети |
| Ollama | бесплатная программа, которая запускает нейросеть на своём компьютере |
| Qwen, Llama | названия нейросетей (Qwen — от Alibaba, Llama — от Meta) |
| RAG | «генерация с поиском»: сначала найти тексты, потом ответить по ним |
| HTML, тег | язык, на котором записаны страницы сайтов; тег — служебная пометка в <угловых скобках> |
| Парсинг | разбор страницы сайта, чтобы достать из неё нужный текст |
| Карточка (чанк) | небольшой кусочек статьи, около полстраницы |
| Стоп-слова | частые слова без смысловой нагрузки: «кто», «в», «на» |
| Стемминг, основа | отрезание окончания: «журавлях» → «журавл» |
| BM25 | формула поиска, в которой редкие слова важнее частых |

| Слово | Простыми словами |
|---|---|
| Векторный поиск | поиск по смыслу с помощью «координат смысла» (в проекте не используется) |
| Промпт | текст задания для нейросети |
| Токен | кусочек текста (обычно часть слова), которым «думает» нейросеть |
| Temperature | насколько случайно нейросеть выбирает слова; 0 — без случайности |
| Гипотеза | предположение, которое проверяют экспериментом |
| Ключевое слово | слово из правильного ответа, по которому программа проверяет ответ |

# Вопросы для самопроверки

**1. Почему нейросеть может уверенно ответить неправильно?**

Ответ: Она угадывает следующее слово и обучена писать гладко, а не проверять факты. Если о теме мало текстов, она сочиняет правдоподобное.

**2. Чем наш помощник отличается от обычного чат-бота?**

Ответ: Он сначала ищет ответ в библиотеке проверенных статей и отвечает только по ним, а ещё показывает источники.

**3. Зачем резать статьи на карточки?**

Ответ: Нейросеть читает за раз немного текста, а в маленьком кусочке легче найти нужный факт.

**4. Зачем нужен стемминг?**

Ответ: Чтобы разные формы одного слова («горянка», «горянке») совпадали при поиске.

**5. Почему в BM25 слово «режиссёр» весит больше, чем «год»?**

Ответ: «Режиссёр» встречается в 9 карточках из 303, а «год» — в 79. Редкое слово точнее указывает на нужную карточку.

**6. Зачем в промпте фраза «Если ответа нет — скажи „В моей библиотеке нет ответа“»?**

Ответ: Чтобы у нейросети был честный выход и она не выдумывала.

**7. Почему помощник ответил, что «Журавли» перевёл Козловский?**

Ответ: Поиск нашёл и правильную карточку, и карточку о «Горянке», где Козловский перевёл поэму; маленькая модель выбрала не ту.

**8. Что такое вопрос-ловушка и зачем он нужен?**

Ответ: Вопрос с ложной предпосылкой (Нобелевская премия). Проверяет, не выдумывает ли нейросеть.

**9. Подтвердилась ли гипотеза?**

Ответ: Да: без библиотеки 0 верных из 12, с библиотекой — 7 из 12, ловушки — 3 из 3.

**10. Что такое векторный поиск и почему его нет в проекте?**

Ответ: Поиск по смыслу через «координаты смысла». Его нет, потому что поиск по словам проще понять и проверить, он быстрее и не требует второй нейросети.